

Lista 2

1. Classificação GOF: Explique a diferença fundamental de intenção entre padrões Criacionais, Estruturais e Comportamentais.

- Os padrões **criacionais** oferecem mecanismos de criação de objetos, os quais aumentam a flexibilidade e reutilização de código já existente.
 - Factory Method; Abstract Factory; Builder; Prototype; Singleton
- Os **estruturais** explicam como compor classes e objetos em estruturas maiores e mais complexas. Focam na organização dos relacionamentos, garantindo que o sistema seja flexível, eficiente e fácil de manter.
 - Adapter; Bridge; Composite; Decorator; Facade; Flyweight; Proxy
- Os **comportamentais** são as soluções focadas em algoritmos, comunicação e delegação de responsabilidades entre objetos. Facilitam a interação em sistemas, promovendo reutilização de código e reduzindo acoplamento.
 - Chain of Responsibility; Command; Iterator; Mediator; Memento; Observer; State; Strategy; Template Method; Visitor

2. Singleton: Por que o padrão Singleton é frequentemente considerado um “anti-padrão” em contextos de testes unitários e sistemas multitarefa?

O Singleton garante que uma classe tenha apenas uma instância, providenciando um ponto de acesso global a ela. É bom em casos como um objeto de banco de dados, onde é usado para evitar múltiplas conexões simultâneas, abertura e fechamento constantes de sessões e para economizar recursos de memória.

No entanto, no desenvolvimento moderno, nem sempre funciona bem. Quando usamos testes unitários, o objetivo é isolar o código e testá-lo independentemente e de forma determinística (a entrada e saída devem sempre ser as mesmas). O Singleton dificulta esse processo, pois os testes irão modificar permanentemente o objeto Singleton. Da próxima vez que o teste for rodado, os resultados serão diferentes.

Em sistemas multitarefa o Singleton é considerado “anti-padrão” porque, neles, várias threads rodam o código simultaneamente. Deve haver um tratamento especial para evitar que 2 ou mais instâncias separadas do objeto Singleton não sejam criadas, quebrando a principal regra do padrão. Também não é muito ideal, pois se várias threads precisarem ler dados desse Singleton o tempo todo, ficarão presas em filas, diminuindo a escalabilidade e paralelismo do sistema.

3. Factory Method vs. Abstract Factory: Em qual situação você escolheria criar uma “família de objetos relacionados” em vez de apenas um objeto específico?

A diferença do Factory Method pro Abstract Factory é a escala do problema. Enquanto o Factory Method foca na criação de um tipo de objeto apenas, o Abstract Factory é uma classe/interface com métodos de fábricas diferentes, englobando a criação de vários objetos relacionados (família) através de composição, não herança simples.

- Factory Method: **método abstrato** dentro de uma classe, usa **herança**
- Abstract Factory: **interface/classe** para **outras fábricas**, usa **composição**

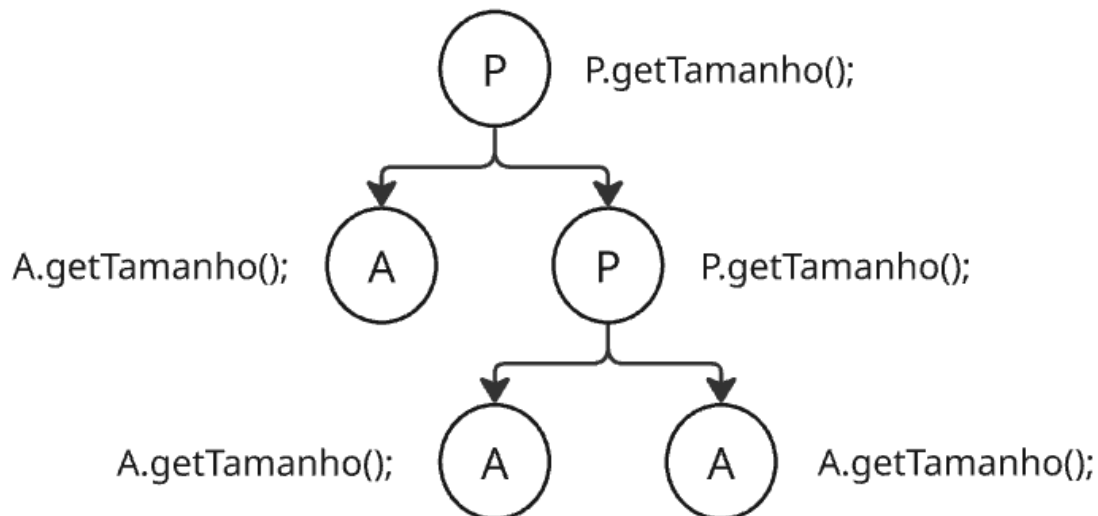
4. Facade: Como o padrão Facade ajuda a reduzir o acoplamento entre um sistema complexo e seus clientes?

O Facade serve para trazer uma interface que abstrai o funcionamento de um sistema complexo, como uma biblioteca/framework. Quem utiliza o Facade não precisa saber os detalhes que o Facade sabe, diminuindo o acoplamento do cliente. O cliente conversa apenas com o Facade, sem chamar nada da biblioteca/framework diretamente.

5. Composite: Como o padrão Composite permite que clientes tratem objetos individuais e composições de objetos de maneira uniforme? (Exemplo: Pastas e Arquivos).

O Composite é usado quando queremos tratar os **objetos individuais** e **coleções de objetos inteiras da mesma forma**. Temos uma interface comum entre pastas e arquivos, que é o que o cliente vê. Se o cliente chama **pasta.getTamanho()** ou **arquivo.getTamanho()**, os dois funcionarão da mesma forma da perspectiva do cliente. No entanto, arquivos e pastas implementam o mesmo método de suas próprias maneiras.

No caso, a pasta é uma coleção de arquivos ou outras pastas (é o objeto composto). A forma que o padrão Composite facilita o processo é que as ações de um objeto Composite são delegadas para os objetos individuais. No exemplo, o **pasta.getTamanho()** é em essência **um loop que chama o .getTamanho() de seus objetos**. Se for um arquivo, ele retornará seu tamanho, mas se for uma outra pasta (subpasta), ela irá também chamar o **.getTamanho()** do que tiver dentro dela.



6. Proxy: Diferencie um Proxy Remoto de um Proxy Virtual (Lazy Loading).

7. Strategy: Como o padrão Strategy permite eliminar grandes blocos de condicionais (if/else ou switch) em um código?

8. Observer: Explique o fluxo de notação no padrão Observer (Subject e Observers). Como ele implementa o conceito de Loose Coupling?

9. State: No padrão State, quem é responsável pela transição de um estado para outro: o Contexto ou as subclasses de Estado? Justifique.

10. Visitor: Por que o padrão Visitor é útil para adicionar operações a estruturas de objetos complexas sem alterar as classes desses objetos?

O padrão Visitor é usado quando queremos adicionar uma operação nova sobre uma estrutura de classes que já existe e não podemos alterar. Em vez de colocar a lógica da operação nova dentro das classes já existentes, criamos um objeto externo (Visitor) que irá percorrer cada elemento dessa estrutura executando a tarefa desejada.

Por exemplo, temos uma estrutura que representa os componentes de um computador: classes Processador, PlacaMae e MemoriaRam. O cliente pede pra criar uma funcionalidade de **gerar relatório de preço** de cada peça. Em outro momento, ele pede pra criar um **relatório de consumo de energia**. Sem o visitor, teríamos que abrir cada classe, adicionar o método **calcularPreco()**, depois abrir tudo de novo e adicionar o **calcularConsumo()**. Isso violaria o OCP, pois teríamos que mudar as classes antes estáveis toda vez que vier um requisito novo.

Com o Visitor, só precisamos criar um método **accept(Visitor visitor)** dentro de cada classe **UMA ÚNICA VEZ**. Toda a lógica da visita vai para dentro de um ConcreteVisitor, que recebe um objeto do tipo Processador, PlacaMae ou MemoriaRam e realiza as operações necessárias.

- 11. Refactoring - Extract Method: Quando e por que devemos aplicar a refatoração “Extrair Método” em um código longo?**
- 12. Refactoring - Replace Conditionals: Como o polimorfismo pode ser usado para refatorar expressões condicionais complexas baseadas em “tipos”?**
- 13. Arquitetura em Camadas: Qual é a regra de ouro da comunicação entre camadas (quem pode conhecer quem) para evitar dependências circulares?**
- 14. MVC: Explique o papel de cada componente no MVC. Por que o Controller não deve conter regras de negócio (domínio)?**
- 15. Microsserviços: Quais são as principais vantagens e os principais desafios (como transações distribuídas) ao migrar de um Monolito para Microsserviços?**
- 16. Pub/Sub vs. Message Queues: Diferencie o modelo de entrega de mensagens “Ponto a Ponto” (Filas) do modelo “Publicação/Assinatura”.**
- 17. Verificação vs. Validação: No contexto de QA, explique a frase: “Estamos construindo o produto corretamente? vs. Estamos construindo o produto correto?”.**
- 18. TDD (Test Driven Development): Descreva o ciclo “Red-Green-Refactor” e explique por que a refatoração é um passo obrigatório no processo.**
- 19. Cobertura de Código: Ter 100% de cobertura de código garante que o software está livre de bugs? Justifique sua resposta.**
- 20. Testes de Unidade vs. Integração: Qual a importância do uso de Mocks e Stubs em testes de unidade para garantir o isolamento do componente testado?**